RallyUp

-An AI-Powered Tennis Social and Matching Platform

Abstract

RallyUp addresses the difficulty tennis players face in finding compatible partners, coordinating schedules, and maintaining engagement. The platform combines a rule-based matching engine (skill, location, time preferences, and play intentions) with AI-driven recommendations using multiple LLMs (OpenAI GPT-4o and OpenRouter models: Grok 4 Fast, Llama 3.3 70B Instruct, Gemini 2.0 Flash Experimental, Qwen 2.5 72B Instruct). The frontend is built with Next.js (App Router), React, TypeScript, and Tailwind; the backend uses Django REST Framework with JWT authentication. Core features include Find a Partner (Algorithm vs AI modes with side-by-side comparison), Friends and Chat (bubble layout, token auto-refresh), an AI Assistant, and a Calendar that records exact play time ranges and generates weekly/monthly summaries (total days/hours, max streak). Experiments demonstrate that LLM recommendations improve interpretability and handle heterogeneous preferences better than rules alone, while rule-based matching offers lower cost and stable throughput. The key conclusion is that combining fast, predictable rule-based filtering with explainable AI ranking yields higher match quality and trust, improving user interaction and retention while keeping costs manageable.

Introduction

Problem:

Finding tennis partners with compatible skill, location, and availability is time-consuming; coordination and communication are fragmented.

Objective:

Deliver a unified platform for matching, messaging, scheduling, and progress tracking, while introducing explainable AI recommendations at controlled cost.

Contributions:

Dual-path matching: rule-based similarity scoring and multi-LLM AI recommendation, with a Compare mode. Besides, an AI Tennis ChatBot to answer any questions related to tennis.

Practical product features:

Protected routes, login-aware homepage, chat UX design, AI Assistant, token auto-refresh.

Scheduling analytics:

Calendar with start/end time entries; weekly/monthly aggregates and max streak.

Data

For this project, we constructed a synthetic dataset of **1,000 seed users** to evaluate and compare matchmaking strategies across multiple models. All data were generated programmatically using a custom Django management command, ensuring controlled variability and realistic user distributions. The dataset simulates a real-world tennis matching environment by capturing demographic attributes, tennis skill-related characteristics, personal preferences, and behavioral intentions.

Dataset Source and Size:

The dataset is generated entirely in-house using a Python script within the Django backend (`seed_users` command).

- **Total users:** 1,000
- **Associated profiles:** 1,000
- **Data structure:** Each user has a `User` entry plus a corresponding `Profile` entry.

This synthetic dataset allows reproducibility, consistency, and safe experimentation without involving any real user data.

Main Dataset Characteristics:

Each user profile includes the following feature groups:

1. Demographics
   - **Display name:** Randomized from a list of common first names.
   - **Age:** Uniform distribution between 16–65.
   - **Gender:** {M, F, O}.
   - **Location:** One of ten major U.S. cities (e.g., San Francisco, Chicago, Miami).
2. Tennis Skill & Experience
   - **Skill level:** 1.0 to 5.5 in increments of 0.5 (approximating NTRP scale).
   - **Years playing:** Derived from age minus a random offset, ensuring coherence.
   - **Dominant hand:** {R, L}.
   - **Backhand type:** {1H, 2H}.
3. Match Preferences

Each preference is generated as a random subset:

- **Preferred court types:** {hard, clay, grass}.
- **Match types:** {singles, doubles}.
- **Play intentions:** {casual, competitive}.
- **Preferred languages:** {en, zh}.

This combination yields rich, multi-dimensional profiles, enabling meaningful testing across rule-based and LLM-based matchmaking algorithms.

Preprocessing, Cleaning, and Data Handling:

Although the dataset is synthetic, several preprocessing steps ensure structure and consistency:

1. Validation
   - Avoids duplicated usernames by checking existence before creation.
   - Ensures all categorical features fall within predefined domain sets.
2. Normalization
   - Skill levels are normalized into a consistent scale (1.0–5.5).
   - Categorical lists (e.g., preferred court types) are stored as arrays for easier downstream parsing.
3. Structural Consistency
   - Each user automatically receives a linked `Profile`.
   - Years playing are adjusted to non-negative values (e.g., younger players).
4. Transformation for Modeling

Before feeding the data into matchmaking algorithms:

- Lists are embedded or expanded into binary indicators.
- Text fields (display names, locations) are optionally vectorized for LLM-based methods.
- Skill/experience fields are standardized when needed.

Example Record:

username: seed_user_0999
display_name: Reese 999
age: 24
gender: M
location: Los Angeles
skill_level: 3.0
years_playing: 6
dominant_hand: L
backhand_type: 2H
preferred_court_types: ["hard"]
preferred_match_types: ["doubles"]
play_intentions: ["casual"]
preferred_languages: ["en", "zh"]
The example highlights diversity across skill, geography, experience, preferences, and intentions—mirroring realistic tennis communities.

Simplified Data Schema for Seed Users:

```
+-------------+                +----------------+
|   User      | 1-to-1 ---> |    Profile     |
+-------------+                +----------------+
| id          |                | id             |
| username    |                | user_id (FK)   |
| email       |                | display_name   |
| password    |                | age            |
+-------------+                | gender         |
                               | location       |
                               | skill_level    |
                               | years_playing  |
                               | dominant_hand  |
                               | backhand_type  |
                               | court_types[]  |
                               | match_types[]  |
                               | intentions[]   |
                               | languages[]    |
                               +----------------+
```

Diagram

Experimental Setup

Stack:

- Frontend: Next.js (App Router), React, TypeScript, Tailwind, TanStack Query.
- Backend: Django + DRF + JWT; Nginx; Let's Encrypt; systemd services; environment-isolated secrets.

LLMs:

OpenAI GPT-4o and OpenRouter models (Grok 4 Fast, Llama 3.3 70B Instruct, Gemini 2.0 Flash Experimental, Qwen 2.5 72B Instruct).

Procedure:

- Rule-based: weighted similarity across skill/age/distance/preferences; return a recommendation.
- AI-based: send compact CURRENT_USER + candidates to selected LLM; enforce JSON output; record usage (tokens) and latency; fallback on parse errors.

Parameters:

Candidate size N=5–8; compact prompts; optional JSON mode; non-free endpoints on OpenRouter for stability.

Metrics:

Matching Score measures how good the match between users are, the higher score the higher match, based on the fixed algorithm. User satisfaction, add-friend and chat initiation rates, token usage, latency, error rate, acceptance of AI recommendations and explanations.

Results

Screenshots redacted

Multi-model comparison:

1. Summary Comparison Table

Method	Type	Key Characteristics	Advantages	Limitations	Best Use Cases
Algorithm Match (Rule-based)	Deterministic Scoring Algorithm	Uses predefined weights, formulas, and feature-similarity logic	Extremely fast, fully controllable, highly interpretable, zero cost	Cannot understand natural language or nuanced preferences; requires manual tuning	Large-scale, stable, real-time matching with clear rules
GPT-4o	Advanced LLM	Strong semantic reasoning, deep user-preference understanding	Highest-quality recommendations, robust to noise, excellent at implicit preference extraction	Higher cost, slower than lightweight models	High-precision personalized matchmaking, complex preference modeling
Grok 4 Fast	Speed-optimized LLM	Lightweight reasoning tuned for fast responses	Very fast, cost-efficient, suitable for real-time	Weaker deep reasoning, less nuanced personalization	High-frequency matching, instant recommendation systems
Gemini Flash	Ultra-fast LLM	Google's Flash inference, optimized for speed and scale	Fastest among all models, low cost, efficient for brief inputs	Limited deep reasoning, weaker long-context preference modeling	Large-volume matchmaking, cost-sensitive scenarios

Llama 3.3 70B	Open-source LLM	Strong reasoning, flexible customization	Very low cost (self-hosted), high interpretability, fully controllable	Requires hosting hardware, speed depends on deployment	Customizable matching logic, privacy-sensitive systems
Qwen 2.5 72B Instruct	Open-source LLM (Chinese-optimized)	Excellent Chinese understanding, stable generation	Best performance for Chinese users, highly cost-effective, easy to fine-tune	Slightly weaker deep reasoning than other LLMs	Chinese user base, preference extraction from Chinese descriptions

2. Feature-by-Feature Comparison

Dimension	Algorithm Match	GPT-4o	Grok 4 Fast	Gemini Flash	Llama 3.3 70B	Qwen 2.5 72B
Semantic Understanding	❌	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐ (Strong for Chinese)
Reasoning Depth	❌	⭐⭐⭐⭐	⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐
Speed	⭐⭐⭐⭐⭐	⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	⭐⭐	⭐⭐
Cost	Free	High	Low	Very low	Very low	Very low
Interpretability	⭐⭐⭐⭐⭐	⭐⭐	⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐
Control & Determinism	⭐⭐⭐⭐⭐	⭐⭐	⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐
Implicit Preference Modeling	Weak	⭐⭐⭐⭐⭐	⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐
Personalization Quality	Low	⭐⭐⭐⭐⭐	⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐
Chinese Language Handling	-	⭐⭐⭐⭐	⭐⭐	⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐

Compare mode:

Improves user understanding and engagement by contrasting algorithmic vs AI choices.

Discussion

Interpretation:

Combining deterministic filters with explainable AI matching balances throughput and quality; users respond positively to reasons behind recommendations.

Limitations:

LLM rate limits and quotas; prompt length drives token costs; cold-start profiles reduce match stability; need robust fallback and caching.

Unexpected findings:

Non-skill factors (availability windows...) substantially affect satisfaction, validating multi-attribute matching.

Cold start:

Not many users in the community, and users with blank tennis profile.

Potential Future Directions

Prompt/structure optimization:

JSON mode/tools, Top-1 outputs with rationale and difference highlights; further field compaction and token control.

Personalized weights:

Learn preference weights from user actions to dynamically adapt rule-based scoring.

Social graph signals:

"friends of friends," venue/time networks to enrich recommendations.

Map Integration:

Connect players via an interacted map to let users see the locations and distance on the map. Besides, more functions like searching the closest public court.

A/B testing:

Quantitatively measure model impact on conversion/retention.

Production hardening:

Rate limiting, quotas, observability, error budgets, auto-rollback.

Mobile and notifications:

Push reminders integrated with calendar and chat.

Conclusion

RallyUp integrates rule-based similarity matching with multi-LLM recommendations to deliver a practical tennis social platform spanning matching, chat, scheduling, and analytics. AI explanations may meaningfully improve trust and match satisfaction, while rules ensure predictable cost and performance. The key takeaway is that a dual-path strategy, fast filtering plus explainable ranking, provides superior user experience and scalable operations. Continued work will focus on personalization, social graph integration, and production-grade governance.